

Python Tip: Starred Unpacking

Need to unpack a list?

But don't know exactly how many values it contains?

Use `*` to capture the flexible part!



Variable-Length Data

Frequently in coding, we work with lists where the order of the elements has meaning, but the total length of the list can vary.

```
# A file path can have any number of folders
path_parts = ["users", "ada", "docs", "report.txt"]

# A command can have any number of arguments
command_parts = ["copy", "report.txt", "backup/"]
```

In `path_parts`, the last value is the filename, while the preceding values are the folders.

In `command_parts`, the first value is the command—the action to perform. The remaining values are the arguments it needs.

How can we easily parse these lists?

Don't Parse It by Hand

Indexing and slicing work, but they focus on positions instead of meaning.

```
command_parts = ["copy", "report.txt", "backup/"]  
  
command = command_parts[0]  
arguments = command_parts[1:]  
  
print("Command:", command)  
print("Arguments:", arguments)
```

```
Command: copy  
Arguments: ['report.txt', 'backup/']
```

Python gives us a clearer way to say, “the first value and everything else.”

The Starred Unpacking Rule

Regular variables receive one value. A starred variable captures the remaining values in a list.

```
command_parts = ["copy", "report.txt", "backup/"]  
  
command, *arguments = command_parts  
  
print("Command:", command)  
print("Arguments:", arguments)
```

```
Command: copy  
Arguments: ['report.txt', 'backup/']
```

The assignment now describes the meaning of the data directly.

The Star Can Move

For a file path, the filename is last while the number of folders can vary.

```
path_parts = "users/ada/docs/report.txt".split("/")  
  
*folders, filename = path_parts  
  
print("Folders:", folders)  
print("File:", filename)
```

```
Folders: ['users', 'ada', 'docs']  
File: report.txt
```

The starred variable captures the flexible beginning instead.

Unpack Into Functions

We can also use `*` to unpack a list into a function call.

```
def copy_file(source, destination):  
    print(source, "->", destination)  
  
command_parts = ["copy", "report.txt", "backup/"]  
command, *arguments = command_parts  
  
copy_file(*arguments)
```

```
report.txt -> backup/
```

On the left of `=`, `*` gathers values. In a function call, it spreads them.

Wrap-Up

Use starred unpacking when positions have meaning, but part of a sequence can vary in length.

`command, *arguments` captures everything after the first value.

`*folders, filename` captures everything before the last value.

`function(*arguments)` spreads those values into a function call.

Follow me for more tips.

Shep Bryan IV